

Lab 5 — Secure File Sharing

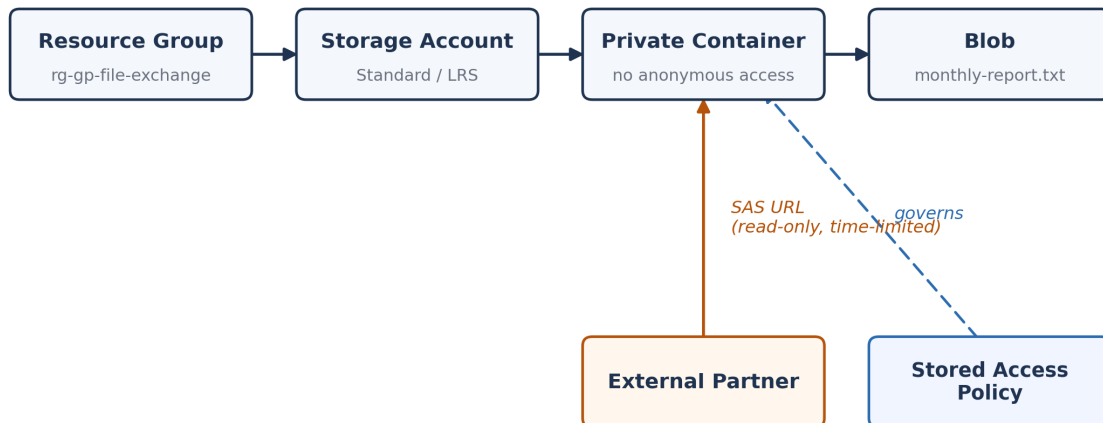
Azure Blob Storage · SAS · Stored Access Policies

Sharing a file with an external partner through Azure Blob Storage — privately, with read-only, time-limited access, and no exposed credentials.

rg-gp-file-exchange · stgpfilexchg65553815 · partner-drop · Standard / LRS

Objective & Architecture

This lab sets up a private Azure Blob Storage container and shares a single file with an external partner — without a public link, without handing over the storage account's key, and with access that can be revoked at any time. Sharing is controlled through a stored access policy and a read-only, time-limited Shared Access Signature (SAS).



The partner reaches the blob only through a signed SAS URL into the private container, governed by a stored access policy. The container sits inside the storage account and resource group hierarchy.

Azure Storage Setup

A Standard-performance, locally redundant (LRS) storage account was provisioned inside a dedicated resource group, with a private container for the shared file — anonymous access disabled at the account level.

The screenshot shows the 'Create a storage account' page in the Azure portal. The breadcrumb navigation at the top reads 'Home > Storage center | Blob Storage'. The page title is 'Create a storage account'. The 'Resource group' is set to 'rg-gp-file-exchange'. Under 'Instance details', the 'Storage account name' is 'stgpfilerchg65553815', the 'Region' is '(Europe) North Europe', and the 'Primary service' is 'Azure Blob Storage or Azure Data Lake Storage'. For 'Performance', the 'Standard' option is selected. For 'Redundancy', 'Locally redundant storage (LRS)' is selected. At the bottom, there are 'Previous', 'Next', and 'Review + create' buttons, along with a 'Give feedback' link.

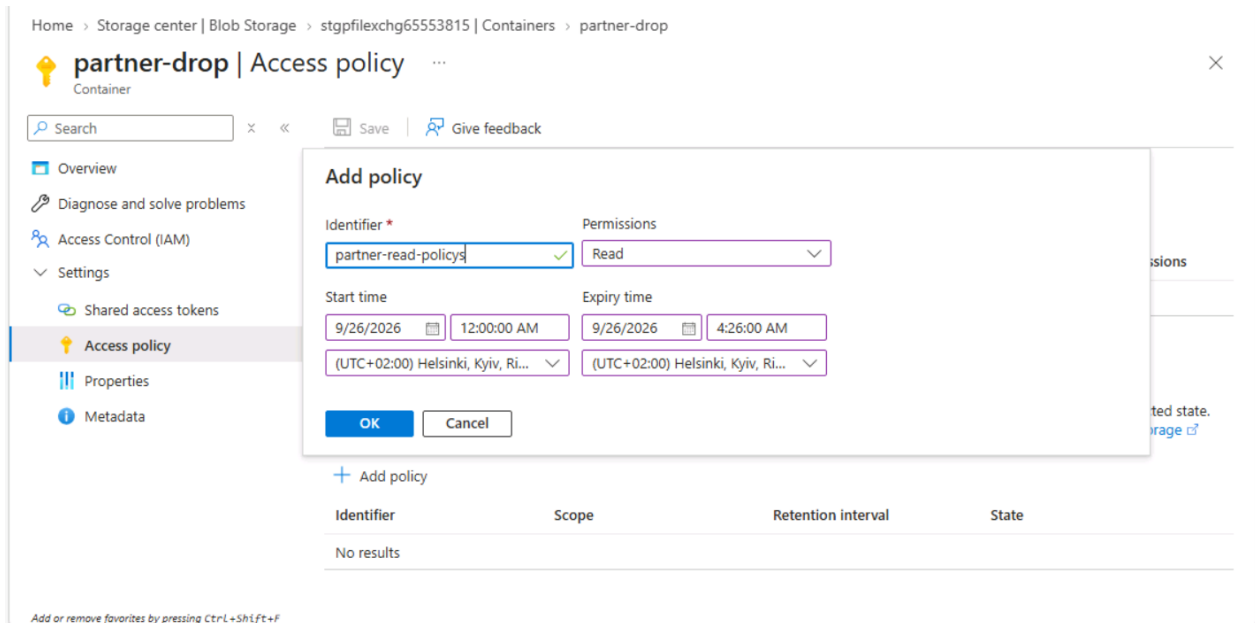
Storage account **stgpfilerchg65553815** in resource group **rg-gp-file-exchange** — Standard performance, locally redundant storage (LRS).

The screenshot shows the 'Containers' page for the storage account 'stgpfilerchg65553815'. The left sidebar lists various services, with 'Containers' selected. The main area shows a table with one container, 'partner-drop', created on 9/26/2026. A 'New container' dialog is open on the right, showing the 'Name' field with 'partner-drop' and the 'Anonymous access level' set to 'Private (no anonymous access)'. A message states: 'The access level is set to private because anonymous access is disabled on this storage account.' The 'Create' button is at the bottom of the dialog.

Container **partner-drop** created private — anonymous access disabled at the account level.

Secure File Sharing

The file (**monthly-report.txt**, a sample file created for this lab) was shared through a Shared Access Signature rather than a public link or the storage account's key. The SAS references a stored access policy (**partner-read-policy**) that scopes it to read-only permissions within a fixed validity window — so access can be revoked centrally at any time, without rotating the storage account's keys.



Stored access policy defining read-only, time-limited access for the SAS.

Troubleshooting

The SAS URL initially returned an **AuthenticationFailed** error. I checked the stored access policy and found I'd entered the validity time incorrectly. After correcting the policy's time window and generating the SAS again, the file became accessible.

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="utf-8"?>
<Error>
  <Code>AuthenticationFailed</Code>
  <Message>Server failed to authenticate the request. Make sure the value of Authorization header is formed correctly including the signature.
  RequestId:cd9adfea-801e-0037-36a2-4d4a8b000000 Time:2026-09-26T10:34:47.1578042Z</Message>
  <AuthenticationErrorDetail>Signature not valid in the specified time frame: Start [Fri, 25 Sep 2026 21:00:00 GMT] - Expiry [Sat, 26 Sep 2026 01:26:00 GMT] - Current [Sat, 26 Sep 2026 10:34:47 GMT]</AuthenticationErrorDetail>
</Error>
```

Initial SAS request failed — AuthenticationFailed.

```
Monthly Partner Report
Status: Complete
Items processed: 142
Storage tier: Standard
Compliance check: Passed
Next review: Scheduled
```

Same request succeeding after correcting the policy's time window.

Lifecycle Management

A lifecycle management rule was added to automatically delete blobs under the shared path after 30 days, keeping the container from accumulating old shared files.

Home > Storage center | Blob Storage > stgpfilexchg65553815 | Lifecycle management

Add a rule

×

1 Details2 Base blobs3 Filter set

A rule is made up of one or more conditions and actions that apply to the entire storage account. Optionally, specify that rules will apply to particular blobs by limiting with filters.

Rule name *

delete-shared-files

Rule scope *

☐ Apply rule to all blobs in your storage account

☒ Limit blobs with filters

Blob type *

☒ Block blobs

☐ Append blobs

Blob subtype *

Previous

Next

Lifecycle rule **delete-shared-files**, scoped to block blobs via a filter rather than the whole account.

Security

- Anonymous/public access is disabled, so the blob cannot be accessed through an unsigned public request.
- Read-only, time-limited SAS — the partner receives no standing credential.
- Stored access policy — permissions and expiry are centrally controlled and revocable at any time.
- No storage account keys, connection strings, or full SAS URLs are shared with the partner or exposed in this document.
- Lifecycle management automatically cleans up shared files after 30 days.

Production Perspective

This is a training lab rather than a production implementation. In a production environment, SAS generation would typically be handled by an application or controlled workflow, secrets would be protected through services such as Azure Key Vault, and storage redundancy and monitoring would be selected according to business requirements.

What I Learned

- How Azure Blob Storage separates account-level and container-level access controls.
- How stored access policies scope and centralize control over SAS permissions and expiry.
- How to generate and use a SAS for external, read-only file sharing.
- How to diagnose and fix an AuthenticationFailed SAS error.
- How lifecycle management rules automate blob cleanup.